

Interpolation search

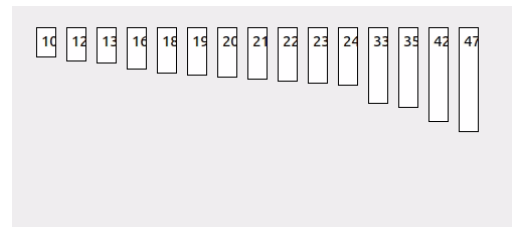
Interpolation search is an algorithm for searching for a key in an array that has been ordered by numerical values assigned to the keys (*key values*). It was first described by W. W. Peterson in 1957.^[1] Interpolation search resembles the method by which people search a telephone directory for a name (the key value by which the book's entries are ordered): in each step the algorithm calculates where in the remaining search space the sought item might be, based on the key values at the bounds of the search space and the value of the sought key, usually via a linear interpolation. The key value actually found at this estimated position is then compared to the key value being sought. If it is not equal, then depending on the comparison, the remaining search space is reduced to the part before or after the estimated position. This method will only work if calculations on the size of differences between key values are sensible.

By comparison, binary search always chooses the middle of the remaining search space, discarding one half or the other, depending on the comparison between the key found at the estimated position and the key sought — it does not require numerical values for the keys, just a total order on them. The remaining search space is reduced to the part before or after the estimated position. The linear search uses equality only as it compares elements one-by-one from the start, ignoring any sorting.

On average the interpolation search makes about $\log(\log(n))$ comparisons (if the elements are uniformly distributed), where n is the number of elements to be searched. In the worst case (for instance where the numerical values of the keys increase exponentially) it can make up to $O(n)$ comparisons.

In interpolation-sequential search, interpolation is used to find an item near the one being searched for, then linear search is used to find the exact item.

Interpolation search



Visualization of the interpolation search algorithm in which 24 is the target value.

<u>Class</u>	<u>Search algorithm</u>
<u>Data structure</u>	<u>Array</u>
<u>Worst-case performance</u>	<u>$O(n)$</u>
<u>Best-case performance</u>	<u>$O(1)$</u>
<u>Average performance</u>	<u>$O(\log(\log(n)))$^[2]</u>
<u>Worst-case space complexity</u>	<u>$O(1)$</u>
<u>Optimal</u>	Yes

Performance

Using big-O notation, the performance of the interpolation algorithm on a data set of size n is $O(n)$; however under the assumption of a uniform distribution of the data on the linear scale used

for interpolation, the performance can be shown to be $O(\log \log n)$.^{[3][4][5]}

Dynamic interpolation search extends the $O(\log \log n)$ bound to other distributions and also supports $O(\log n)$ insertion and deletion.^{[6][7]}

Practical performance of interpolation search depends on whether the reduced number of probes is outweighed by the more complicated calculations needed for each probe. It can be useful for locating a record in a large sorted file on disk, where each probe involves a disk seek and is much slower than the interpolation arithmetic.

Index structures like B-trees also reduce the number of disk accesses, and are more often used to index on-disk data in part because they can index many types of data and can be updated online. Still, interpolation search may be useful when one is forced to search certain sorted but unindexed on-disk datasets.

Adaptation to different datasets

When sort keys for a dataset are uniformly distributed numbers, linear interpolation is straightforward to implement and will find an index very near the sought value.

On the other hand, for a phone book sorted by name, the straightforward approach to interpolation search does not apply. The same high-level principles can still apply, though: one can estimate a name's position in the phone book using the relative frequencies of letters in names and use that as a probe location.

Some interpolation search implementations may not work as expected when a run of equal key values exists. The simplest implementation of interpolation search won't necessarily select the first (or last) element of such a run.

Book-based searching

The conversion of names in a telephone book to some sort of number clearly will not provide numbers having a uniform distribution (except via immense effort such as sorting the names and calling them name #1, name #2, etc.) and further, it is well known that some names are much more common than others (Smith, Jones,) Similarly with dictionaries, where there are many more words starting with some letters than others. Some publishers go to the effort of preparing marginal annotations or even cutting into the side of the pages to show markers for each letter so that at a glance a segmented interpolation can be performed.

Sample implementation

The following C++ code example is a simple implementation. At each stage it computes a probe position then as with the binary search, moves either the upper or lower bound in to define a

smaller interval containing the sought value. Unlike the binary search which guarantees a halving of the interval's size with each stage, a misled interpolation may reduce/i-case efficiency of $O(n)$.

```

1  import <cassert>;
2
3  import std;
4
5  using std::vector;
6
7  /*
8   arr[low, high) is sorted, search the data "key" in this array,
9   if "key" is found, return the corresponding index (NOT necessarily the highest possible index);
10  if "key" is not found, just return low - 1
11
12  How to verify that the algorithm is correct?
13  Proof:
14  (finiteness: after one loop, the width of [low, high] decreases strictly )
15
16  Fist, high <--- high - 1
17  scenario 1. when low = high
18  scenario 2. when low < high, arr[low] = arr[high]
19  scenario 3. when low < high, arr[low] < arr[high], key < arr[low] or key > arr[high]
20  scenario 4. when low < high, arr[low] < arr[high], arr[low] <= key <= arr[high]
21
22  Now let's analyze scenario 4:
23  Once entering the "while" loop, low <= middle <= high
24
25
26  let's analyse after one loop(if we don't return), whether "low > high" will occur
27  After one loop:
28      case a1: "low" branch has been executed in this loop
29          arr[middle] < key <= arr[high]
30          so we have middle < high
31          so after this loop, we have
32              low <= high
33      case a2: "high" branch has been executed in this loop
34          arr[low] <= key < arr[middle]
35          so we have low < middle
36          so after this loop, we have
37              low <= high
38
39  so after one loop(if we don't return), we have "low <= high"
40
41
42  when we exit the "while" loop:
43      case b1: arr[low] >= arr[high]
44          In the last loop, if "low" branch is executed, we know
45              arr[low - 1] < k <= arr[high]
46              arr[low] >= arr[high]
47              low <= high
48              so we have
49              arr[low - 1] < k <= arr[low] = arr[high]
50          In the last loop, if "high" branch is executed, we know
51              arr[low] <= key < arr[high + 1]
52              arr[low] >= arr[high]
53              low <= high
54              so we have
55              arr[low] = arr[high] <= key < arr[high + 1]
56
57      case b2: (arr[low] < arr[high]) && (arr[low] > key):
58          In the last loop, "low" must have been changed
59          so we have
60              arr[low - 1] < key
61          so we have
62              arr[low - 1] < key < arr[low]
63
64      case b3: (arr[low] < arr[high]) && (key > arr[high])
65          In the last loop, "high" must have been changed
66          so we have
67

```

```
68         key < arr[high + 1]
69         so we have
70         arr[low] < arr[high] < key < arr[high + 1]
71
72     */
73
74     // version 1
75     template <typename T>
76     static Rank interpolationSearch(vector<T>& arr, const T& key, Rank low, Rank high) {
77         high -= 1;
78         int middle;
79         int initialLow = low;
80
81         while ((arr[low] < arr[high]) && (arr[low] <= key) && (key <= arr[high])) {
82             middle = low + ((key - arr[low]) * (high - low)) / (arr[high] - arr[low]);
83
84             assert((low <= middle) && (middle <= high));
85
86             if (arr[middle] < key) {
87                 low = middle + 1;
88             } else if (key < arr[middle]) {
89                 high = middle - 1;
90             } else {
91                 return middle;
92             }
93         }
94
95         if (key == arr[low]) {
96             return low;
97         } else {
98             return initialLow - 1;
99         }
100     }
101
102     /*
103     search "key" in the sorted array arr[low, high),
104     return: the highest index i such that arr[i] <= key
105
106     How to verify that the algorithm is correct?
107     Proof:
108     finiteness: after one loop, the width of [low, high] decreases strictly
109
110     Fist, high <---- high - 1
111     scenario 1. when low = high
112     scenario 2. when low < high, key < arr[low] or arr[high] <= key
113     scenario 3. when low < high, arr[low] <= key < arr[high]
114
115     Now let's analyze scenario 3:
116     Once entering the "while" loop, low <= middle < high
117
118     when we exit the "while" loop:
119     case a1: key < arr[low]
120         so "low" is changed in the last loop, we know
121         arr[low - 1] <= key < arr[low]
122     case a2: arr[high] <= key
123         so "high" is changed in the last loop, we know
124         key < arr[high], impossible
125
126     conclusion: we should return "low - 1"
127
128     */
129
130     // version 2
131     template <typename T>
132     static Rank interpolationSearch(vector<T>& arr, const T& key, Rank low, Rank high) {
133         high -= 1;
134         assert(low <= high);
135         Rank middle;
136
137         if (key < arr[low]) {
138             return low - 1;
139         }
```

```

140     }
141     if(arr[high] <= key) {
142         return high;
143     }
144     // now low < high , arr[low] <= key < arr[high]
145     while ((arr[low] <= key) && (key < arr[high])) {
146         middle = low + ((high - low) * (key - arr[low])) / (arr[high] - arr[low]);
147
148         assert((low <= middle) && (middle < high));
149
150         if(key < arr[middle]) {
151             high = middle;
152         } else {
153             low = middle + 1;
154         }
155     }
156
157     return low - 1;
158 }

```

Notice that having probed the list at index *mid*, for reasons of loop control administration, this code sets either *high* or *low* to be not *mid* but an adjacent index, which location is then probed during the next iteration. Since an adjacent entry's value will not be much different, the interpolation calculation is not much improved by this one step adjustment, at the cost of an additional reference to distant memory such as disk.

Each iteration of the above code requires between five and six comparisons (the extra is due to the repetitions needed to distinguish the three states of $<$ $>$ and $=$ via binary comparisons in the absence of a three-way comparison) plus some messy arithmetic, while the binary search algorithm can be written with one comparison per iteration and uses only trivial integer arithmetic. It would thereby search an array of a million elements with no more than twenty comparisons (involving accesses to slow memory where the array elements are stored); to beat that, the interpolation search, as written above, would be allowed no more than three iterations.

See also

- Linear search
- Binary search
- Interpolated binary search,^[8] a hybrid of interpolated search and binary search
- Exponential search
- Ternary search
- Hash table
- Newton's method
- Flashsort, using the distribution of values for sorting rather than searching

References

- W. W. Peterson (1957). "Addressing for Random-Access Storage". *IBM J. Res. Dev.* **1** (2): 130–

146. doi:10.1147/rd.12.0130 (<https://doi.org/10.1147%2Frd.12.0130>).
2. Simon Yuan. "Understanding The Complexity Of Interpolation Search, Seminar Advanced Algorithms and Data Structures" (<https://cadmo.ethz.ch/education/lectures/HS18/SAADS/reports/17.pdf>) (PDF).
3. Weiss, Mark Allen (2006). *Data structures and problem solving using Java*, Pearson Addison Wesley
4. Armenakis, A. C., Garey, L. E., Gupta, R. D., An adaptation of a root finding method to searching ordered disk files, BIT Numerical Mathematics, Volume 25, Number 4 / December, 1985.
5. Sedgewick, Robert (1990), *Algorithms in C*, Addison-Wesley
6. Mehlhorn, Kurt; Tsakalidis, Athanasios (1993). "Dynamic interpolation search". *Journal of the ACM*. **40** (3): 621–634. doi:10.1145/174130.174139 (<https://doi.org/10.1145%2F174130.174139>). ISSN 0004-5411 (<https://search.worldcat.org/issn/0004-5411>).
7. Andersson, Arne; Mattsson, Christer (1993). "Dynamic interpolation search in $o(\log \log n)$ time". *Automata, Languages and Programming*. Vol. 700. Berlin, Heidelberg: Springer Berlin Heidelberg. p. 15–27. doi:10.1007/3-540-56939-1_58 (https://doi.org/10.1007%2F3-540-56939-1_58). ISBN 978-3-540-56939-8.
8. Mohammed, Adnan Saher; Amrahov, Şahin Emrah; Çelebi, Fatih V. (1 October 2021). "Interpolated binary search: An efficient hybrid search algorithm on ordered datasets" (<http://www.sciencedirect.com/science/article/pii/S221509862100046X>). *Engineering Science and Technology*. **24** (5): 1072–1079. doi:10.1016/j.jestch.2021.02.009 (<https://doi.org/10.1016%2Fj.jestch.2021.02.009>). ISSN 2215-0986 (<https://search.worldcat.org/issn/2215-0986>)
- .

External links

- Interpolation search (<http://www.dcc.uchile.cl/~rbaeza/handbook/algs/3/322.srch.p.html>)
 - National Institute of Standards and Technology (<https://xlinux.nist.gov/dads/HTML/interpolationSearch.html>)
 - Interpolation Search - A Log LogN Search (<https://www.cs.technion.ac.il/~itai/publications/Algorithms/p550-perl.pdf>)
-

Retrieved from "https://en.wikipedia.org/w/index.php?title=Interpolation_search&oldid=1332633084"